

Absolutely. Below is a **100-question AI Engineering interview set** based directly on the skills you provided. I also added dedicated questions about **LangGraph** and **smolagents**, especially around agents, state management, workflows, tools, and production design.

I kept the answers in **simple English**, so they are easier to understand and use during a real interview.

AI Engineering Interview — 100 Questions & Answers

1. AI Engineering Fundamentals

1. What is AI Engineering?

Answer:

AI Engineering is the process of building software applications that use existing AI models. It includes prompt engineering, API integration, RAG, agents, tool calling, embeddings, vector databases, evaluation, and production deployment.

2. What is the difference between AI Engineering and Machine Learning Engineering?

Answer:

Machine Learning Engineering often focuses on training, improving, and deploying ML models. AI Engineering usually focuses on using existing models such as LLMs to build useful software applications.

3. What is an LLM?

Answer:

LLM stands for Large Language Model. It is a model trained on a large amount of text that can understand and generate natural language. Examples include GPT models and open-source models from Hugging Face.

4. What is an AI-powered application?

Answer:

It is a normal software application that uses AI to perform tasks such as text generation, classification, search, summarization, question answering, or automation.

5. What are the main components of an LLM application?

Answer:

Common components are an LLM, prompts, application logic, tools or APIs, databases, possibly a vector database, conversation state, evaluation, and monitoring.

6. What is the difference between an LLM and an AI agent?

Answer:

An LLM mainly generates responses. An AI agent can use an LLM to decide what actions to take, call tools, observe results, and continue working toward a goal.

7. What is inference?

Answer:

Inference is the process of sending input to a trained model and getting an output from it.

8. What is a foundation model?

Answer:

A foundation model is a large general-purpose model trained on a large dataset. It can be adapted for many different tasks.

9. What is multimodal AI?

Answer:

Multimodal AI can work with more than one type of data, such as text, images, audio, or video.

10. What is the typical architecture of an AI application?

Answer:

A common architecture is:

Client → API → Application Service → LLM → Tools/RAG/Database → Response

The application service controls the workflow and connects the LLM to external systems.

2. OpenAI API and Responses API

11. How do you call an OpenAI model from Python?

Answer:

```
from openai import OpenAI
```

```
client = OpenAI()
```

```
response = client.responses.create(
```

```
model="gpt-5",  
input="Explain RAG in simple words."  
)  
  
print(response.output_text)
```

The application sends a request to the API and processes the returned response.

12. What is the Responses API?

Answer:

The Responses API is an API for building modern AI applications. It supports model responses, tools, structured outputs, multimodal input, streaming, and agent-like workflows.

13. What is the difference between a user message and a system instruction?

Answer:

A system instruction defines how the model should behave. A user message contains the actual request from the user.

For example:

System: You are a helpful customer support agent.

User: I cannot log into my account.

14. What is multi-turn conversation?

Answer:

A multi-turn conversation contains multiple messages exchanged between the user and the AI. The application needs to preserve relevant conversation state so the model can understand previous messages.

15. Why is conversation history important?

Answer:

Without relevant history, the model may not understand references to previous messages. For example, if the user says "What about the second option?", the application needs previous context.

16. What is temperature?

Answer:

Temperature controls how much randomness is used when generating output. Lower values usually produce more consistent output, while higher values can produce more varied output.

17. What is a token?

Answer:

A token is a small piece of text processed by an LLM. A token can be a word, part of a word, punctuation, or another text unit.

18. Why should we care about token usage?

Answer:

Token usage affects cost, latency, and context size. Large prompts and long conversation histories can increase both cost and response time.

19. What is streaming?

Answer:

Streaming means the application receives the model's output gradually instead of waiting for the complete response. It is useful for chat applications because users can see the answer as it is generated.

20. What is tool calling?

Answer:

Tool calling allows the model to request an external function. For example, the model can call a weather API, database function, or payment service.

21. Why should tool execution be controlled by the application?

Answer:

The LLM should not directly control sensitive systems. The application should validate the tool request, check permissions, validate parameters, execute the tool, and return the result.

22. What is structured output?

Answer:

Structured output means asking the model to return data in a predefined format, such as JSON.

For example:

```
{  
  "name": "John",  
  "age": 30  
}
```

This makes the output easier and safer for software to process.

23. Why is structured output better than parsing normal text?

Answer:

Normal text can have many formats. Structured output gives the application a predictable schema, which reduces parsing errors.

24. How do you handle API errors?

Answer:

I use exception handling, retries where appropriate, timeouts, logging, and clear error responses. I also distinguish between temporary errors and permanent errors.

25. How do you reduce OpenAI API costs?

Answer:

I can reduce unnecessary context, use smaller models when possible, cache results, limit output length, improve prompts, and avoid repeated API calls.

3. Prompt Engineering

26. What is prompt engineering?

Answer:

Prompt engineering is the process of designing instructions that help an LLM produce reliable and useful results.

27. What makes a good prompt?

Answer:

A good prompt clearly defines the task, context, constraints, expected output, and important rules.

28. What is zero-shot prompting?

Answer:

Zero-shot prompting means asking the model to perform a task without providing examples.

Example:

Classify this text as positive or negative.

29. What is one-shot prompting?

Answer:

One-shot prompting provides one example before asking the model to perform the task.

30. What is few-shot prompting?

Answer:

Few-shot prompting provides several examples to show the model the expected behavior.

31. What are delimiters in prompts?

Answer:

Delimiters clearly separate different parts of a prompt.

Example:

Summarize the following text:

```
<document>  
...  
</document>
```

They help the model understand the structure of the input.

32. What is role-based prompting?

Answer:

It defines a role for the model, such as:

You are a customer support assistant.

This gives the model behavioral context.

33. What is prompt injection?

Answer:

Prompt injection happens when untrusted input tries to change the instructions given to the model.

For example, a document might contain:

Ignore all previous instructions.

The application should treat external content as untrusted data.

34. How can you reduce prompt injection risks?

Answer:

I separate instructions from user data, validate tool calls, limit permissions, treat retrieved documents as untrusted content, and use application-level authorization.

35. What is prompt refinement?

Answer:

Prompt refinement means testing a prompt, analyzing its results, identifying problems, and improving the prompt iteratively.

36. What is chain-of-thought prompting?

Answer:

It is a prompting technique that asks a model to reason through a problem step by step. In production, I generally prefer asking for the required result or a concise explanation rather than requiring hidden reasoning.

37. What is self-consistency?

Answer:

Self-consistency generates multiple possible solutions and compares them to find a consistent answer. It can improve some reasoning tasks but increases cost and latency.

38. What is conditional prompting?

Answer:

Conditional prompting changes the instructions based on the input.

For example:

If the user asks about billing, use the billing instructions.

If the user asks about shipping, use the shipping instructions.

39. What is context engineering?

Answer:

Context engineering is the broader process of deciding what information should be given to the model, when it should be given, and how it should be structured.

It includes prompts, conversation history, retrieved documents, tool results, user information, and system instructions.

40. What is the difference between prompt engineering and context engineering?

Answer:

Prompt engineering focuses mainly on instructions. Context engineering focuses on the entire information given to the model.

4. Embeddings and Vector Databases

41. What is an embedding?

Answer:

An embedding is a numerical representation of data, usually text. Similar meanings usually produce vectors that are close to each other.

42. Why do we use embeddings?

Answer:

We use embeddings for semantic search, recommendations, document retrieval, classification, clustering, and RAG.

43. What is cosine similarity?

Answer:

Cosine similarity measures how similar two vectors are based on their direction.

A higher value usually means the vectors have more similar meaning.

44. What is semantic search?

Answer:

Semantic search searches based on meaning instead of only matching exact words.

For example, a search for "How can I reset my password?" can find a document saying "Steps to recover your account credentials."

45. What is a vector database?

Answer:

A vector database stores vectors and allows us to efficiently search for similar vectors.

Examples include Pinecone, Weaviate, Milvus, and pgvector.

46. What is metadata filtering?

Answer:

Metadata filtering allows us to filter vectors before or during similarity search.

For example:

department = "finance"

language = "en"

47. Why combine metadata filtering with vector search?

Answer:

Vector similarity finds semantically similar content, while metadata filtering restricts the search to the correct data.

48. What is dimensionality reduction?

Answer:

Dimensionality reduction converts high-dimensional vectors into fewer dimensions so they can be easier to visualize or analyze.

49. What is t-SNE?

Answer:

t-SNE is a dimensionality reduction technique often used to visualize high-dimensional data in two or three dimensions.

50. What is vector similarity search?

Answer:

It searches for vectors that are closest to a query vector according to a similarity metric such as cosine similarity.

5. RAG

51. What is RAG?

Answer:

RAG stands for Retrieval-Augmented Generation. It retrieves relevant information from an external knowledge source and gives that information to the LLM before generating an answer.

52. Why use RAG?

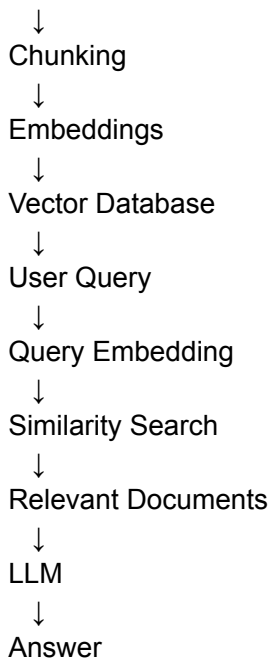
Answer:

RAG allows an application to use private, recent, or domain-specific information without retraining the model.

53. What is a typical RAG pipeline?

Answer:

Documents



54. What is document chunking?

Answer:

Chunking means splitting large documents into smaller pieces before creating embeddings.

55. Why is chunking important?

Answer:

If chunks are too large, retrieval may contain too much irrelevant information. If they are too small, important context can be lost.

56. What is chunk overlap?

Answer:

Chunk overlap means repeating a small part of one chunk in the next chunk. It helps preserve context between chunks.

57. What is a RAG hallucination?

Answer:

It happens when the model generates information that is not supported by the retrieved data.

58. How can you improve RAG quality?

Answer:

I can improve chunking, embeddings, retrieval, metadata filtering, query rewriting, reranking, context formatting, and evaluation.

59. What is reranking?

Answer:

Reranking takes the initial search results and uses another model or algorithm to reorder them based on relevance.

60. How do you evaluate a RAG system?

Answer:

I evaluate retrieval quality and answer quality separately.

For retrieval, I can use metrics such as recall and precision. For answers, I can evaluate correctness, relevance, faithfulness, and citation accuracy.

6. Pinecone

61. What is Pinecone?

Answer:

Pinecone is a vector database designed for storing and searching vector embeddings.

62. What is an index in Pinecone?

Answer:

An index is a collection used to store and search vectors.

63. What is upsert?

Answer:

Upsert means inserting a new vector or updating an existing vector with the same ID.

64. What information should you store with a vector?

Answer:

Usually I store the vector, a unique ID, and metadata such as document ID, source, category, language, and permissions.

65. How would you build semantic search with Pinecone?

Answer:

I would generate an embedding for each document, store the vectors in Pinecone, embed the user query, search for similar vectors, apply metadata filters if needed, and return the relevant documents.

7. Hugging Face

66. What is Hugging Face?

Answer:

Hugging Face is a platform and ecosystem for AI models, datasets, libraries, and tools. It provides many open-source models for NLP, vision, audio, and other tasks.

67. What is the Hugging Face Hub?

Answer:

The Hugging Face Hub is a repository where developers can find and share models, datasets, and other AI resources.

68. What is Transformers?

Answer:

Transformers is a popular Hugging Face library for working with transformer-based models.

69. What is a tokenizer?

Answer:

A tokenizer converts text into tokens that a model can process.

70. What is a Transformers pipeline?

Answer:

A pipeline provides a simple interface for common AI tasks.

Example:

```
from transformers import pipeline
```

```
classifier = pipeline("sentiment-analysis")  
result = classifier("I love this product.")
```

71. What is zero-shot classification?

Answer:

Zero-shot classification classifies text into labels without training a specific classifier for those labels.

72. What is the difference between local inference and hosted inference?

Answer:

Local inference runs the model on our own infrastructure. Hosted inference uses an external provider.

Local inference gives more control, while hosted inference is usually easier to operate.

8. LangChain

73. What is LangChain?

Answer:

LangChain is a framework for building applications around LLMs. It provides components for prompts, models, tools, retrievers, agents, chains, and document processing.

74. What is a chain in LangChain?

Answer:

A chain is a sequence of operations where the output of one step becomes the input of another step.

75. What is a retriever?

Answer:

A retriever finds relevant documents or information based on a query.

76. What is a prompt template?

Answer:

A prompt template is a reusable prompt with dynamic variables.

Example:

Answer the question using this context:

{context}

Question:

{question}

77. How does LangChain help with RAG?

Answer:

LangChain provides components for document loading, text splitting, embeddings, vector stores, retrievers, prompts, and LLM calls.

78. What is memory in an LLM application?

Answer:

Memory is a mechanism for maintaining useful information from previous interactions.

It can include conversation history, summaries, user preferences, or stored facts.

9. LangGraph

79. What is LangGraph?

Answer:

LangGraph is a framework for building stateful, graph-based AI workflows and agents. It is useful when an application needs multiple steps, state, branching, loops, human approval, or tool calls.

80. Why would you use LangGraph instead of a simple LangChain chain?

Answer:

A simple chain works well for a fixed sequence. LangGraph is better when the workflow has branching, loops, state, retries, human intervention, or multiple agents.

81. What is a node in LangGraph?

Answer:

A node represents a step in the workflow. It can call an LLM, execute a tool, validate data, retrieve documents, or perform another operation.

82. What is an edge in LangGraph?

Answer:

An edge defines how the workflow moves from one node to another.

For example:

User → Agent → Tool → Agent → Final Answer

83. What is state in LangGraph?

Answer:

State contains the information shared between different nodes in the graph.

For example:

```
{
  "messages": [],
  "user_id": 123,
  "search_results": []
}
```

84. Why is state important in agent workflows?

Answer:

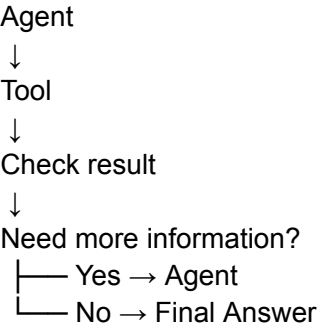
Agents often need information from previous steps. State allows different nodes to access and update that information.

85. How can LangGraph handle loops?

Answer:

The graph can move from one node back to a previous node based on a condition.

For example:



86. How would you implement human approval in LangGraph?

Answer:

I would pause the workflow before a sensitive action, ask a human for approval, store the decision in the state, and then continue or stop the workflow.

87. What is a good use case for LangGraph?

Answer:

A good use case is an AI customer-support agent that can search documentation, check an order, ask for additional information, escalate to a human, and then continue the workflow.

10. smolagents

88. What is smolagents?

Answer:

smolagents is a lightweight framework from Hugging Face for building AI agents. It focuses on simple agent implementations and allows agents to use tools to perform actions.

89. What is the main idea behind smolagents?

Answer:

The main idea is to make agent development simple. Instead of building a large framework, we can define tools and allow the model to use them to solve tasks.

90. What is a tool in smolagents?

Answer:

A tool is a function that an agent can use to interact with the outside world.

For example:

```
def get_weather(city: str):  
    ...
```

The agent can decide when to use this function.

91. What is the difference between smolagents and LangGraph?

Answer:

smolagents focuses on lightweight agent creation and tool use. LangGraph focuses more on controlling complex, stateful workflows and graphs.

They can solve similar problems but have different design goals.

92. When would you choose LangGraph?

Answer:

I would choose LangGraph when I need explicit workflow control, state management, branching, loops, persistence, or human-in-the-loop processes.

93. When would you choose smolagents?

Answer:

I would choose smolagents when I need a lightweight agent with tools and do not need a complex workflow engine.

11. MCP

94. What is Model Context Protocol?

Answer:

MCP is a protocol that standardizes how AI applications connect models to external tools, resources, and data sources.

95. What are the main components of MCP?

Answer:

The main components include:

- MCP Host
 - MCP Client
 - MCP Server
 - Tools
 - Resources
 - Prompts
-

96. What is an MCP server?

Answer:

An MCP server exposes tools and resources that an AI application can use.

For example, an MCP server could expose tools for querying a database or reading files.

97. What is an MCP tool?

Answer:

An MCP tool is an operation that an AI application can call.

For example:

```
get_customer(id)
create_order(data)
search_products(query)
```

98. Why is MCP useful?

Answer:

MCP creates a standard way to connect AI applications with external systems. This can reduce custom integration code and make tools reusable across AI clients.

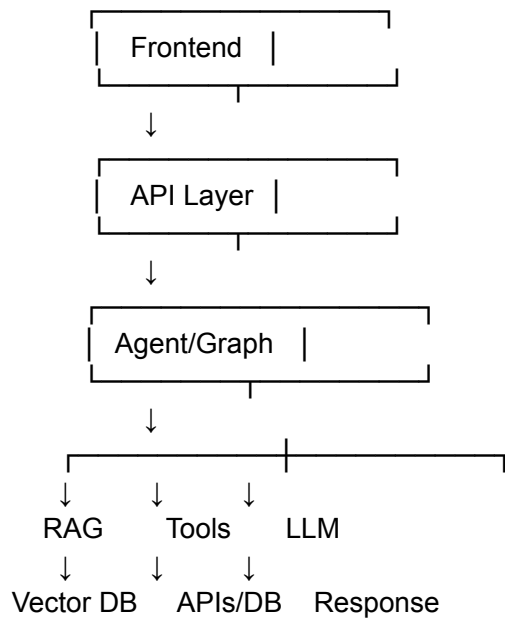
99. How would you secure an MCP server?

Answer:
I would use authentication, authorization, input validation, parameterized database queries, timeouts, rate limits, logging, and least-privilege access.

I would also carefully review third-party MCP servers before connecting them to sensitive systems.

100. Design an AI agent that can answer questions, search documents, and perform actions. How would you build it?

Answer:
I would design it as several components:



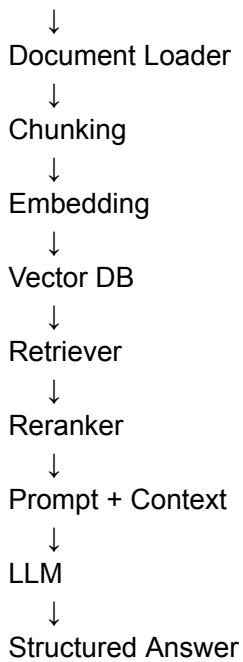
I would use **RAG** for knowledge retrieval, **tool calling** for external actions, and **LangGraph** if the workflow requires state, branching, loops, or human approval. MCP could expose external tools in a standard way. I would use structured outputs for reliable communication between the LLM and application code.

For production, I would also add authentication, authorization, validation, logging, tracing, retries, timeouts, evaluation, monitoring, and cost control.

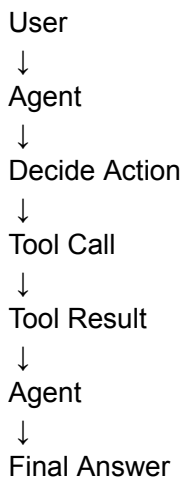
Bonus: Topics You Should Be Ready to Explain in a Real Interview

Based on the skills you listed, I would expect a **Senior/AI Engineer interview** to go beyond definitions. Interviewers may ask you to design or implement things such as:

RAG System
PDF/Web Data



Agent System



Production AI Architecture

You should be comfortable discussing:

- Prompt versioning
- Model selection
- Model fallback
- Token/cost management
- Streaming
- Structured outputs
- Tool calling
- Function validation
- RAG
- Embedding strategies
- Chunking
- Reranking
- Conversation memory
- Agent state
- LangGraph
- MCP

- Evaluation
- Hallucination detection
- Prompt injection
- Data privacy
- Authentication
- Authorization
- Rate limiting
- Retries
- Timeouts
- Caching
- Observability
- Logging
- Tracing
- Unit testing
- Integration testing
- LLM evaluation
- Human-in-the-loop
- Production deployment

For your interview preparation, the most important part is not memorizing these 100 answers. You should be able to take a question such as *"Design an AI customer-support system"* and connect **OpenAI** → **prompt/context** → **RAG** → **vector DB** → **tool calling** → **agent** → **LangGraph/MCP** → **evaluation** → **production architecture** into one coherent design.

AI agent and agentic AI are related but not the same thing. The short version:

- **AI agent** = a *thing* (a system/software entity that perceives, decides, acts). is a software program designed to perform specific tasks without intention, with a degree of autonomy
- **Agentic AI** = a *property or paradigm* (AI that behaves with agency—goal-directed, autonomous, proactive). A Framework where multiple AI Agents can collaborate and make decisions independently to achieve a higher goal

Agent component

1. Node = python function -> task
2. Edges = connect the nodes and Conditional Edge
3. State , State Schema : serves as the input for all the nodes
4. State Graph : structure of the entire graph

There is two main pipeline in RAG:

- Data injection pipeline
- Retrieval pipeline